

12 3 2025 section's task report

```
!pip install torch_geometric

import torch
from torch_geometric.data import Data
from torch_geometric.nn import SAGEConv
import torch.nn.functional as F

# 0,1,1,1]
```

- The first line installs the torch_geometric library, and the rest of lines import torch and torch.nn.functional for build a Graph Neural Network.

```
#--- Define a small graph with 6 nodes ---
# Node features (2 features per node).
# Here benign users have [1, 0] and malicious have [0, 1] for illustration.
x = torch.tensor(
    [
        [1.0, 0.0], # Node 0 (benign)
        [1.0, 0.0], # Node 1 (benign)
        [1.0, 0.0], # Node 2 (benign)
        [0.0, 1.0], # Node 3 (malicious)
        [0.0, 1.0], # Node 4 (malicious)
        [0.0, 1.0] # Node 5 (malicious)
    ],
    dtype=torch.float,
)
```

- This code defines the features of each node in the graph with each node representing a user.
- Number of nodes = 6 (Node 0 to Node 5)
- Each node has two features and each feature is flaot
- [1.0, 0.0] : Benign user (safe/normal)
- [0.0, 1.0] : Malicious user (harmful)

```

# Edge list (undirected). Connect benign users (0-1-2 fully connected)
# and malicious users (3-4-5 fully connected), plus one cross-edge 2-3.
edge_index = (
    torch.tensor(
        [
            [0, 1],
            [1, 0],
            [1, 2],
            [2, 1],
            [0, 2],
            [2, 0],
            [3, 4],
            [4, 3],
            [4, 5],
            [5, 4],
            [3, 5],
            [5, 3],
            [2, 3],
            [3, 2], # one connection between a benign (2) and malicious (3)
        ],
        dtype=torch.long,
    )
    .t()
    .contiguous()
)

```

- This code defines the graph structure using the edge_index list.
- Here the graph is undirected because each edge is listed in both directions.

```

y = torch.tensor([0, 0, 0, 1, 1, 1], dtype=torch.long)

data = Data(x=x, edge_index=edge_index, y=y)

```

- It determine the true labels for each node (ground truth labels).
 - It create a PyTorch Geometric Graph object containing:
- X : Node features.
edge_index : Graph structure.
y : Labels.

```
class GraphSAGENet(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super(GraphSAGENet, self).__init__()
        self.conv1 = SAGEConv(in_channels, hidden_channels)
        self.conv2 = SAGEConv(hidden_channels, out_channels)

    def forward(self, x, edge_index):
        # First layer: sample neighbors and aggregate
        x = self.conv1(x, edge_index)
        x = F.relu(x) # non-linear activation
        # Second layer: produce final embeddings/class scores
        x = self.conv2(x, edge_index)
        return F.log_softmax(x, dim=1) # log-probabilities for classes
```

- It create a Graph Neural Network using GraphSAGE.
- GraphSAGE is designed to handle Graph data.
- GraphSAGE performs aggregation
- This model has two layers (2-layer).
- in_channels :the number of features of each node
- hidden_channels: the number of hidden embeddings in the first layer.
- out_channels: Output of the second layer (Number of classes)
- def forward : it determine how data is passed across the network.
- conv1:For each node, it aggregates the features of its neighbors and combines them with the features of the node itself.
- ReLU:it is a nonlinear function and helps the network learn nonlinear relationships between nodes.
- conv2:Transforms hidden embeddings to scores for each class (2 classes).
- log_softmax:it gives the log-probabilities for each node in both classes.

```

# Instantiate model: input dim=2, hidden=4, output dim=2 (benign vs malicious)
model = GraphSAGENet(in_channels=2, hidden_channels=4, out_channels=2)

# Simple training loop
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
model.train()
for epoch in range(50):
    optimizer.zero_grad()
    out = model(data.x, data.edge_index)
    loss = F.nll_loss(out, data.y) # negative log-likelihood
    loss.backward()
    optimizer.step()

```

```
model = GraphSAGENet(in_channels=2, hidden_channels=4, out_channels=2)
```

- it create a version of the GraphSAGE model
- in_channels=2 : Each node has 2 attributes
- hidden_channels=4 : The hidden layer represents each node in 4 dimensions
- out_channels=2 : The model predicts whether a node is benign or malicious

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

- Adam is an algorithm for updating the network weights during training.
- model.parameters() : All the weights in the network that will change.
- lr=0.01 : Learning rate.
- model.train() : it specifies that the model is in training mode.
- for epoch in range(50) : it repeats the training 50 times.
- optimizer.zero_grad() : it removes any gradients from the previous step.
- out = model(data.x, data.edge_index) : The model predicts the results (log probabilities for each node).
- loss = F.nll_loss(out, data.y) : it calculates the loss between the predictions and the actual classifications:
- nll_loss = Negative Log-Likelihood loss (suitable for log_softmax).
- loss.backward() : it calculates the gradients for each weight in the grid.
- optimizer.step() : it updates the grid weights based on the gradients and learning rate.

```
# After training, we can check predictions
model.eval()
pred = model(data.x, data.edge_index).argmax(dim=1)
print("Predicted labels:", pred.tolist()) # e.g. [0,0,
```

```
Predicted labels: [0, 0, 0, 1, 1, 1]
```

- The model is in evaluation mode.
- Here we see the model's performance after training.
- Predicted labels: [0, 0, 0, 1, 1, 1]:
This means that nodes 0, 1, and 2 were classified as benign, and nodes 3, 4, and 5 were classified as malicious and This means also the network has learned to differentiate between benign and malicious Even if the features are very basic.